

Sesión 5: Estructuras de Datos No Lineales

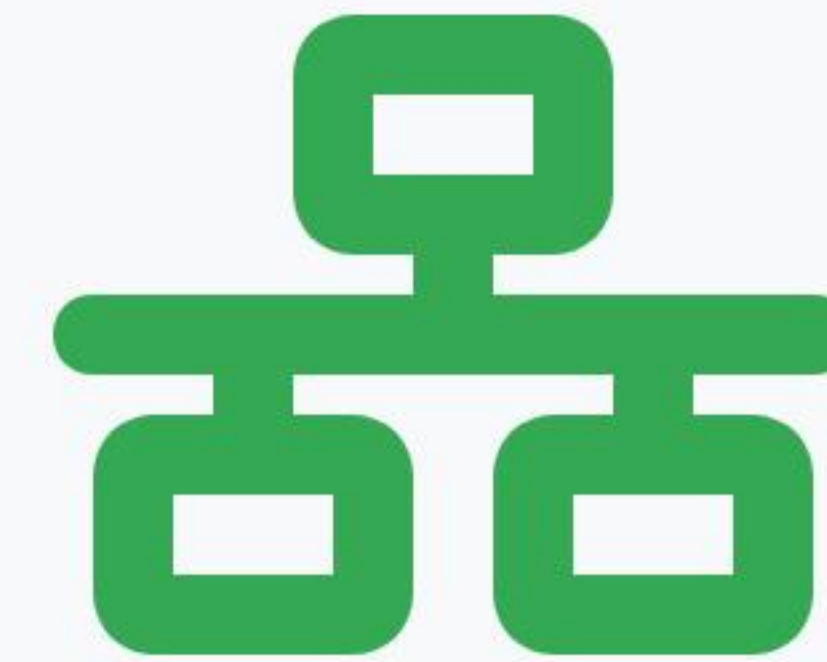
Árboles Binarios de Búsqueda (BST) y Balanceo Exponencial AVL en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

Agenda de Hoy

-  Concepto de estructuras jerárquicas
-  Árboles Binarios de Búsqueda (BST) y recorridos
-  El problema de la degeneración y la solución AVL
-  Rotaciones simples y dobles de balanceo en C#



Eficiencia Logarítmica

Búsquedas ultrarrápidas de tipo $O(\log n)$ mediante ramificación binaria balanceada.

Estructuras No Lineales

Superando la unidireccionalidad secuencial para modelar jerarquías

Concepto de Árbol Jerárquico

A diferencia de las listas o pilas, un árbol representa una relación ****jerárquica uno-a-muchos****. Es una estructura acíclica conectada por aristas.

Raíz (Root)

El nodo origen supremo de la estructura. No posee padres.

Hijos y Padres

Nodos derivados de un ancestro inmediato superior.

Hojas (Leaves)

Nodos terminales de las ramas que apuntan a null.

Árbol Binario de Búsqueda (BST)

La regla de ordenamiento posicional izquierda-derecha

Propiedad Clave del BST

Un Árbol Binario restringe a cada padre a tener un ****máximo de 2 hijos**** (Izquierdo y Derecho). Un BST añade la regla fundamental de ordenación:



Hijo Izquierdo < PADRE < Hijo Derecho

Definición del Nodo de Árbol en C#

Utilizamos una estructura POO que bifurca su puntero referencial en dos direcciones:

```
public class NodoArbol {  
    public int Valor { get; set; }  
    public NodoArbol Izquierdo { get; set; }  
    public NodoArbol Derecho { get; set; }  
  
    public NodoArbol(int valor) {  
        this.Valor = valor;  
        Izquierdo = Derecho = null;  
    }  
}
```


Inserción Recursiva en un BST

```
public NodoArbol Insertar(NodoArbol raiz, int valor) {  
    if (raiz == null) return new NodoArbol(valor);  
  
    if (valor < raiz.Valor)  
        raiz.Izquierdo = Insertar(raiz.Izquierdo, valor); // Mutación a la izquierda  
    else if (valor > raiz.Valor)  
        raiz.Derecho = Insertar(raiz.Derecho, valor); // Mutación a la derecha  
  
    return raiz;  
}
```


Recorridos del Árbol (Transversales)

Al no ser lineales, existen tres formas canónicas fundamentales y sistemáticas de recorrer todos los nodos de forma recursiva:

Pre-Orden

$\text{Raíz} \rightarrow \text{Izq} \rightarrow \text{Der}$

Útil para clonar u obtener una copia exacta del árbol.

In-Orden

$\text{Izq} \rightarrow \text{Raíz} \rightarrow \text{Der}$

¡Mágico! Despliega los elementos ordenados
ascendentemente de forma estricta.

Post-Orden

$\text{Izq} \rightarrow \text{Der} \rightarrow \text{Raíz}$

Utilizado para liberar memoria de abajo hacia arriba
(destrucción).

Implementación del Recorrido In-Orden

La simplicidad y elegancia de la recursividad aplicada a las estructuras de ramificación en .NET:

```
public void InOrden(NodoArbol raiz) {  
    if (raiz != null) {  
        InOrden(raiz.Izquierdo); // Inspecciona subárbol izquierdo  
        Console.Write(raiz.Valor + " "); // Procesa la raíz actual  
        InOrden(raiz.Derecho); // Inspecciona subárbol derecho  
    }  
}
```

El Problema de la Degeneración

Por qué un BST estándar puede fallar catastróficamente en el rendimiento real

Degeneración en Lista Enlazada

Si insertamos datos ordenados secuencialmente en un BST estándar (Ej: 10, 20, 30, 40), el árbol se inclinará por completo hacia un único lado.



Pérdida de Eficiencia:

El árbol pierde su propiedad de ramificación y se comporta estructuralmente como una Lista Simple. La velocidad de búsqueda colapsa de un óptimo **$O(\log n)$** a un ineficiente **$O(n)$** .

Árboles Auto-Balanceados AVL

Adelphi, Velskii y Landis (1962): Control estricto de alturas estructurales

¿Qué es un Árbol AVL?

Un Árbol AVL es un BST dinámico optimizado que se auto-balancea de forma constante. Mide permanentemente la **altura** de sus ramas intermedias.

Factor de Balanceo (FB):

$$FB = \text{Altura_Subárbol_Izquierdo} - \text{Altura_Subárbol_Derecho}$$

Para asegurar que permanezca perfectamente balanceado, la regla de oro obliga a que en cada nodo el factor esté estrictamente limitado a: **$FB \in \{-1, 0, 1\}$** .

Mecanismo de Salvación: Rotaciones

Cuando ocurre una inserción que desbalancea el Factor de Balanceo fuera del rango permitido (valores de 2 o -2), el algoritmo ejecuta operaciones lógicas de **reestructuración referencial** llamadas **Rotaciones**:

Simple

Rotación Izquierda (LI) o Derecha (DR). Se aplican cuando el desbalanceo ocurre en una línea recta externa continua.

Dobles

Rotación Izquierda-Derecha (LD) o Derecha-Izquierda (DI). Requeridas cuando la deformidad estructural tiene forma de zig-zag interno.

Implementación de Rotación Simple Izquierda

```
private NodoAVL RotarIzquierda(NodoAVL x) {  
    NodoAVL y = x.Derecho;  
    NodoAVL T2 = y.Izquierdo;  
  
    // Ejecución física de la rotación referencial  
    y.Izquierdo = x;  
    x.Derecho = T2;  
  
    // Recalcular alturas de los nodos modificados  
    x.Altura = Max(ObtenerAltura(x.Izquierdo), ObtenerAltura(x.Derecho)) + 1;  
    y.Altura = Max(ObtenerAltura(y.Izquierdo), ObtenerAltura(y.Derecho)) + 1;  
  
    return y; // La nueva raíz del subárbol balanceado  
}
```

Comparativa Asintótica de Búsqueda

El poder real del balanceo automatizado en grandes bases de datos

Matriz Big O: BST vs AVL

Operación	Caso Promedio (BST)	Peor Caso (BST Degenerado)	Peor Caso (Árbol AVL Balanceado)
Búsqueda (Search)	$O(\log n)$	$O(n)$ - Lento	$O(\log n)$ - Garantizado
Inserción (Insert)	$O(\log n)$	$O(n)$ - Lista	$O(\log n)$ - Incluye Rotación
Eliminación (Delete)	$O(\log n)$	$O(n)$ - Costoso	$O(\log n)$ - Estable

📊 Para 1,000,000 de registros: $O(n)$ realiza **1,000,000 comparaciones**, mientras que un AVL balanceado $O(\log n)$ lo resuelve en **máximo 20 comparaciones**.

Herramientas de Visualización de Árboles

Técnicas avanzadas para depurar estructuras jerárquicas complejas

Visualización Profesional con Graphviz

Inspeccionar un árbol imprimiéndolo en consola es sumamente complejo.

La práctica estándar en el laboratorio consiste en exportar la estructura a formato **DOT** para renderizar imágenes de diagramas limpios usando la herramienta externa **Graphviz**.

```
// Sintaxis de salida en formato DOT:  
digraph G {  
    50 -> 30;  
    50 -> 70;  
    30 -> 20;  
}
```


Valor de la Unidad: Respeto



"El respeto mutuo entre compañeros de clase, hacia el personal docente y las normativas académicas de la facultad crea el entorno armónico idóneo para el crecimiento de la ingeniería guatemalteca."

— Código de Convivencia Estudiantil USAC

Conclusiones

Jerarquía

Los árboles superan los esquemas lineales permitiendo modelados organizacionales complejos y eficientes.

Estabilidad

El enfoque AVL blindo el rendimiento asintótico impidiendo la degeneración de datos a $O(n)$.

Recursividad

Los recorridos In-Orden, Pre-Orden y Post-Orden demuestran la potencia del control de flujo recursivo.

¿Dudas o Preguntas de la Sesión 5?

¡Muchas gracias por su atención!

✉ Fernando Vicente: 3021894750101@ingenieria.usac.edu.gt
💬 Foro Técnico sobre Estructuras AVL en la plataforma UEDI